PROYECTO UNIDAD II: UTM DESK NOSQL CLOUD

Sistema Integral de Gestión, Resolución y Analítica de Incidencias en el Campus

Asignatura: Bases de Datos en la Nube

Asignatura	Bases de Datos en la Nube
Unidad	II. Bases de datos NoSQL y modelos no relacionales en la nube
Proyecto	UTM Desk NoSQL Cloud: Sistema Integral de Gestión, Resolución y Analítica de Incidencias en el Campus
Modalidad	Proyecto colaborativo en equipos
Tecnologías	MongoDB Atlas y Postman
Entregables	Reportes de avance formativo, repositorio Git, script maestro, PDF (APA 7.ª ed.) y exposición del sistema

Competencia que se evalúa:
Diseñar, implementar, consultar y asegurar una arquitectura de datos no relacional en la nube.

1. Planteamiento de la situación problemática y justificación arquitectónica

1.1 Situación problemática

La operación diaria del campus universitario involucra contingencias que provienen de tres fuentes operativas con necesidades de información dispares:

- **1. Cómputo y Telecomunicaciones:** Fallas en switches de red, puntos de acceso Wi-Fi desconectados, caídas del servidor local o equipos dañados en laboratorios. Requieren registrar direcciones IP, VLANs, puertos de uplink y conteo de equipos afectados.
- **2. Infraestructura Física y Mantenimiento:** Filtraciones en techos, luminarias fundidas, fallas en sistemas de climatización o desperfectos en cerraduras. Exigen documentar tipo de daño, riesgo a la seguridad, requerimiento de insumos o intervención de contratistas externos.
- **3. Sistemas y Servicios Académicos:** Errores de acceso al aula virtual (LMS), problemas para asentar actas en el portal de calificaciones o bloqueo de cuentas de correo institucional. Requieren identificar plataformas, códigos de curso, periodos escolares y cantidad de estudiantes impactados.

La gestión tradicional mediante correos y hojas de cálculo presenta tres limitaciones críticas:

Rigidez e ineficiencia relacional: Cada categoría maneja atributos técnicos dispares. Un motor SQL fuerza la creación de esquemas planos con múltiples columnas nulas o el uso de JOINs costosos para consultar un reporte con sus revisiones y adjuntos.

Trazabilidad dispersa: El seguimiento de revisiones, comentarios de técnicos y evidencias multimedia (imágenes de diagnóstico o reportes PDF) termina separado en carpetas externas sin integridad con el ticket.

Carencia de analítica directiva en tiempo real: Dificultad para calcular los tiempos medios de resolución por edificio y detectar fallas crónicas en aulas específicas sin realizar exportaciones manuales.

2.Justificación de la arquitectura NoSQL Documental

Requerimiento técnico	Motor / Componente	Justificación de ingeniería
Reportes heterogéneos y trazabilidad	MongoDB Atlas M0 (Documental)	El modelo polimórfico almacena la información específica de cada rama dentro del subdocumento detalle_especifico sin desperdiciar almacenamiento. Las estructuras tags, historial_seguimiento y evidencias_adjuntas se embeben en arreglos, permitiendo leer todo el expediente en una sola operación I/O.
Evidencias multimedia	Cloud Storage + BSON	Las capturas y documentos de diagnóstico se alojan en buckets externos; MongoDB Atlas almacena metadatos estructurados y referencias de almacenamiento temporal, garantizando no rebasar el límite de 512 MB de la capa M0.

3. Explicación del sistema y modelado de casos de uso

CU-01: Registrar Incidencia

Actores: Reportante (Docente, Estudiante o Administrativo), Sistema UTM Desk.

Propósito: Dar de alta un ticket técnico adaptando sus propiedades según el origen de la falla sin recurrir a campos nulos.

Precondiciones: El usuario cuenta con una sesión activa validada.

Flujo Principal:

1. El actor envía el reporte con los campos universales: titulo, descripcion, categoria, prioridad, reportante y ubicacion.
2. El actor especifica el subdocumento detalle especifico según la categoría: computo redes, infraestructura o academico.
3. El sistema valida la estructura mediante el validador $jsonSchema de MongoDB Atlas.
4. El sistema ejecuta INCR en la clave de Redis contador:incidencias:YYYY-MM-DD:categoria para obtener el consecutivo diario.

5. Se inserta el documento en incidencias campus con estado inicial abierta y arreglo historial seguimiento inicializado.

Postcondiciones: La incidencia queda almacenada con código formal (ej. INC-2026-0042) y el endpoint retorna HTTP 201 Created.

CU-02: Cargar Evidencia Multimedia

Actores: Reportante o Técnico Operativo, Cloud Storage.

Propósito: Asociar archivos fotográficos o diagnósticos en formato PDF al expediente de la incidencia sin sobrecargar la cuota de almacenamiento.

Precondiciones: La incidencia debe existir en la colección.

Flujo Principal:

1. El actor sube el archivo físico a la capa gratuita de Cloud Storage.
2. El sistema genera una referencia temporal firmada con vigencia acotada.
3. Se realiza una operación de actualización $push en el arreglo evidencias adjuntas guardando: nombre archivo, tipo mime, peso kb, url storage y fecha carga.

CU-03: Actualizar Estado y Bitácora de Seguimiento

Actores: Técnico Asignado, Coordinador de Soporte.

Propósito: Registrar avances técnicos, cambiar el estatus del reporte y acumular horas de trabajo en una sola operación sin condiciones de carrera.

Precondiciones: La incidencia se encuentra en estado abierta, en proceso o en espera.

Flujo Principal:

1. El técnico ingresa el nuevo estado, las horas invertidas en la intervención y un comentario descriptivo.
2. El sistema ejecuta una sentencia atómica updateOne sobre MongoDB Atlas con $set sobre estado (y fecha resolucion si pasa a resuelta), $push en historial seguimiento e $inc en horas invertidas.

CU-04: Gestionar Etiquetas Técnicas Dinámicas

Actores: Técnico Asignado, Coordinador del Área.

Propósito: Categorizar la incidencia con descriptores clave para auditoría y analítica posterior, evitando duplicados.

Flujo Principal:

1. Para agregar etiquetas, el sistema invoca el operador $addToSet sobre el arreglo tags[]. Si el descriptor ya existe, el arreglo permanece intacto.
2. Para retirar una etiqueta errónea, el sistema invoca el operador $pull sobre tags[].

CU-05: Búsqueda Textual y Consulta por Descriptores

Actores: Personal de Soporte, Directivos.

Propósito: Localizar incidencias previas mediante coincidencia sobre el resumen y la descripción.

Flujo Principal:

1. El actor introduce palabras clave.
2. El sistema ejecuta una consulta MQL con el operador $text: { $search: "..." }.
3. MongoDB Atlas evalúa el índice de texto ponderado en español y proyecta el valor score: { $meta: "textScore" }.
4. Los resultados se ordenan de forma descendente conforme a su relevancia.

CU-06: Generar Analítica de Resolución y Fallas Crónicas

Actores: Coordinación de UTM, Dirección Administrativa.

Propósito: Extraer métricas consolidadas sobre el tiempo medio de atención y las causas recurrentes de falla por edificio.

Flujo Principal:

1. Pipeline 1 (Eficiencia operativa por edificio): $match por resuelta/cerrada, cálculo de horas transcurridas, $group por edificio ($avg, $sum, conteo) y $sort descendente.
2. Pipeline 2 (Ranking de tags y problemas recurrentes): $unwind sobre tags, $group por tag, $sort descendente y $limit de 10 elementos.

CU-07: Depurar Incidencias Resueltas por Ciclo de Vida

Actores: Hilo de Mantenimiento Interno de MongoDB Atlas.

Propósito: Purgar de la colección activa aquellos reportes que han alcanzado 60 días naturales desde su fecha de resolución.

Flujo Principal:

1. Al resolverse la incidencia, el sistema asienta el campo fecha resolucion con formato Date.
2. El hilo en segundo plano de MongoDB Atlas elimina los documentos que rebasan la ventana.

4. Requisitos técnicos mínimos de la base de datos

4.1 MongoDB Atlas M0: Colección incidencias campus

Volumen: 20 a 30 incidencias sintéticas distribuidas de forma balanceada: Cómputo y Telecomunicaciones, Infraestructura Física y Mantenimiento, Sistemas y Servicios Académicos.

Diversidad de datos: Al menos 4 edificios o zonas del campus, 3 prioridades (baja, media, alta, critica), 5 estados (abierta, en proceso, en espera, resuelta, cerrada), incidencias recurrentes y mínimo 8 con fecha resolucion.

Polimorfismo: El subdocumento detalle especifico debe adoptar esquemas técnicos heterogéneos según la categoría, sin campos nulos de relleno.

Arreglos y subdocumentos obligatorios: tags[], historial seguimiento[], evidencias adjuntas[] (al menos 15 documentos con evidencias), reportante{} y ubicacion{}.

Índices obligatorios: Compuesto { categoria: 1, estado: 1, "ubicacion.edificio": 1 }, TTL { fecha_resolucion: 1 } (60 días) y Full-Text Search { titulo: "text", descripcion: "text" } ponderado en español.

Operaciones atómicas obligatorias: $set, $inc, $push, $addToSet y $pull.

Pipelines analíticos obligatorios: Pipeline 1 (promedio y total de horas por edificio) y Pipeline 2 (ranking top 10 tags con $unwind).

4.2 Cloud Storage y Postman

Almacenamiento: Imágenes y reportes técnicos en bucket cloud; en MongoDB solo metadatos y referencias de acceso temporal.

Postman: Colección probada con POST (201 Created), GET (200 OK), PATCH/PUT (200 OK) y error de validación (400 Bad Request).

5. Criterio de Selección de Tecnologías

- **Núcleo de Base de Datos Obligatorio: MongoDB Atlas** como repositorio documental principal de incidencias universitarias, justificando el polimorfismo y las estructuras anidadas/arreglos sin valores nulos.
- **Libre Elección del Equipo (justificación técnica en el reporte):**
    - **Backend / API:** Libre elección (Node.js/Express, Python/FastAPI, C#/.NET, Java/Spring Boot, PHP/Laravel, Go, etc.) o integración vía MongoDB Atlas App Services / Data API. El equipo debe fundamentar su elección en experiencia, rendimiento HTTP.
    - **Interfaz de Usuario (UI):** Libre elección para web o móvil (React, Vue, Angular, Svelte, HTML5/Tailwind/Bootstrap, Flutter, React Native, etc.). Se exige que sea intuitiva y funcional para que los usuarios levanten reportes y los técnicos visualicen el avance.
    - **Herramientas de Cliente y Pruebas API:** Postman.
    - **Tecnologías Complementarias:** Caché en memoria o almacenamiento en buckets (Cloud Storage) en capa gratuita.

6. Fases del Proyecto

- **FASE1: Entrega de Avances Técnicos y Presentación al Docente**
    - **Entregable Digital:** Documento formal en formato **PDF** subido a la plataforma institucional con:

1. *Selección justificada de tecnologías:* Explicación técnica de la elección de backend, UI y herramientas.
2. *Requisitos del sistema:* Especificación formal de requisitos funcionales (RF) y no funcionales (RNF).
3. *Modelado formal de datos en JSON/BSON:* Esquema de las colecciones documentando la heterogeneidad de las incidencias (subdocumentos para cada categoría, arreglos de historial/etiquetas y tipos nativos).

- **Dinámica:** Presentación y revisión en clase ante el docente de este documento para recibir retroalimentación formativa y validar la arquitectura técnica antes de la entrega final.

- **FASE 2: Documento Final y Defensa Técnica Integral (Cierre del proyecto)**
  - **Entregable Digital:** Archivo comprimido **.zip** conteniendo:

    1. *Documento Final en PDF (Normas APA 7.ª ed.):* Portada, problemática, justificación, especificación final de requisitos, tecnologías finales utilizadas, capturas nítidas del sistema y de la UI en funcionamiento, esquema JSON final poblado en MongoDB Atlas, banco formal de pruebas en Postman (mostrando URL, método HTTP, cabeceras, código de estado y payload JSON), conclusiones individuales y referencias bibliográficas en formato APA 7.
    2. *Código y Enlaces Cloud:* Script consolidado / JSON exportado, enlace al repositorio Git y URL.
  - **Dinámica de Presentación y Defensa Oral:** Exposición del sistema funcionando en vivo en la nube (MongoDB Atlas + UI + API), demostración de operaciones CRUD y consultas analíticas, concluyendo con la defensa individual respondiendo a las preguntas técnicas del docente para calificar a cada integrante.